

LOAN PREDICTION

NAME: NIKHIL TIWARY

COURSE: ARTIFICIAL INTELLIGENCE

DATE: 30/03/26



INTRODUCTION

•This project focuses on predicting loan repayment behavior using machine learning techniques. Financial institutions face risk when approving loans, and incorrect decisions can lead to losses. This system analyzes borrower details such as age,

loan amount, education, and repayment history to classify whether a loan will be paid or defaulted.

PROBLEM

•Banks struggle with identifying reliable borrowers. Manual decisions are slow and error-prone. This project predicts whether a borrower will repay a loan based on historical data using ML.



OBJECTIVE

•Build a model to predict loan status. Reduce financial risk. Understand factors influencing repayment. Demonstrate ML pipeline including preprocessing, training, and evaluation.

DATASET DESCRIPTION

- Dataset contains 500 records with features like Principal, age, education, gender, past due days, and loan status. Includes date columns used for feature engineering.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv("Loan payments data.csv")
df.head()
```

	Loan_ID	loan_status	Principal	terms	effective_date	due_date	paid_off_time	past_due_days	age	education	Gender
0	xqd20166231	PAIDOFF	1000	30	9/8/2016	10/7/2016	9/14/2016 19:31	NaN	45	High School or Below	male
1	xqd20168902	PAIDOFF	1000	30	9/8/2016	10/7/2016	10/7/2016 9:00	NaN	50	Bechalor	female
2	xqd20160003	PAIDOFF	1000	30	9/8/2016	10/7/2016	9/25/2016 16:58	NaN	33	Bechalor	female
3	xqd20160004	PAIDOFF	1000	15	9/8/2016	9/22/2016	9/22/2016 20:00	NaN	27	college	male
4	xqd20160005	PAIDOFF	1000	30	9/9/2016	10/8/2016	9/23/2016 21:36	NaN	28	college	female


```
print(df.shape)
print(df.columns)
df.info()
```

```
(500, 11)
Index(['Loan_ID', 'loan_status', 'Principal', 'terms', 'effective_date',
      'due_date', 'paid_off_time', 'past_due_days', 'age', 'education',
      'Gender'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Loan_ID         500 non-null   object
1   loan_status     500 non-null   object
2   Principal       500 non-null   int64
3   terms          500 non-null   int64
4   effective_date  500 non-null   object
5   due_date        500 non-null   object
6   paid_off_time   400 non-null   object
7   past_due_days   200 non-null   float64
8   age            500 non-null   int64
9   education       500 non-null   object
10  Gender          500 non-null   object
dtypes: float64(1), int64(3), object(7)
memory usage: 43.1+ KB
```

```
df = df.drop('Loan_ID', axis=1)
df['effective_date'] = pd.to_datetime(df['effective_date'])
df['due_date'] = pd.to_datetime(df['due_date'])
df['paid_off_time'] = pd.to_datetime(df['paid_off_time'])
df['effective_day'] = df['effective_date'].dt.day
df['due_day'] = df['due_date'].dt.day
df['paid_day'] = df['paid_off_time'].dt.day
df['loan_duration'] = (df['due_date'] - df['effective_date']).dt.days    #LOAN DURATION
df = df.drop(['effective_date', 'due_date', 'paid_off_time'], axis=1)
```

```
df.fillna(df.mode().iloc[0], inplace=True)
df['loan_status'] = df['loan_status'].astype('category').cat.codes
```

DATA PREPROCESSING

- Dataset contains 500 records with features like Principal, age, education, gender, past due days, and loan status. Includes date columns used for feature engineering.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

FEATURE ENGINEERING

- Extracted day features and calculated loan duration from date columns to improve model performance.

```
X = df.drop('loan_status', axis=1)
y = df['loan_status']
```

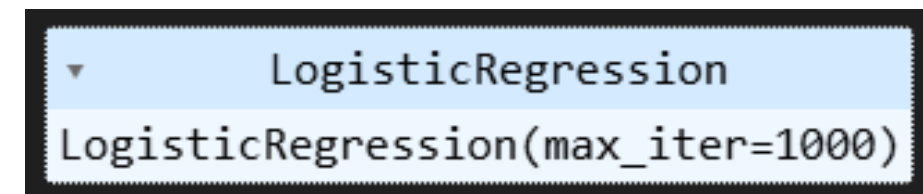
MODEL BUILDING

- Used Logistic Regression. Applied train-test split and scaling. Model learns relationship between features and target.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```



```
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

MODEL EVALUATION

•Used Logistic Regression. Applied train-test split and scaling. Model learns relationship between features and target.

```
from sklearn.metrics import accuracy_score, confusion_matrix
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 0.82

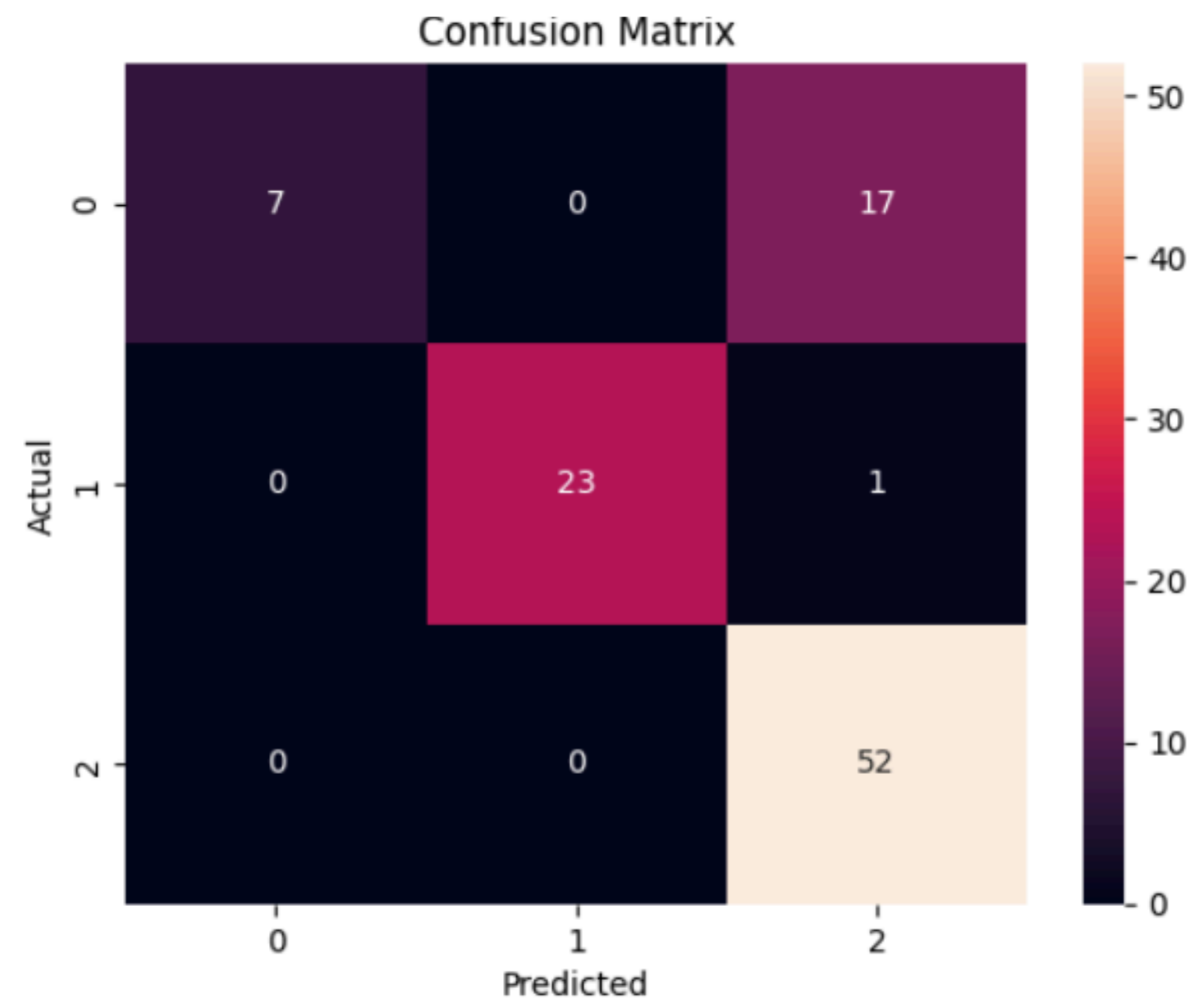
Confusion Matrix:

```
[[13 0 11]
```

```
 [ 0 22 2]
```

```
 [ 5 0 47]]
```

```
import seaborn as sns
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



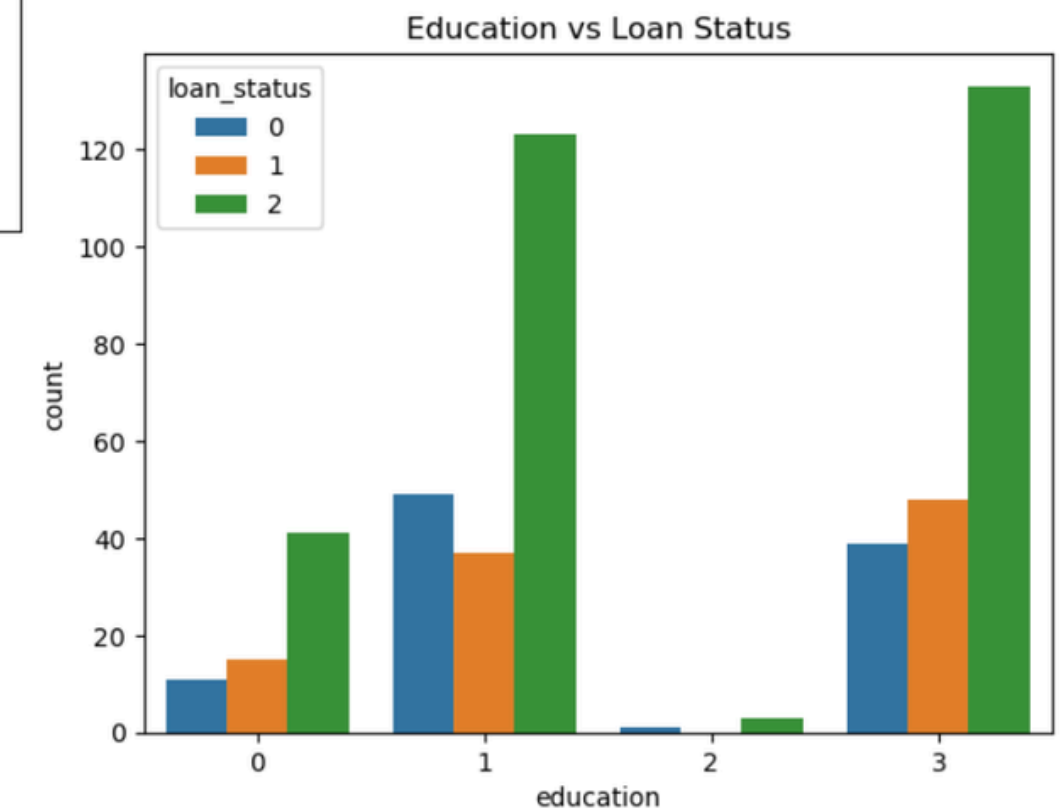
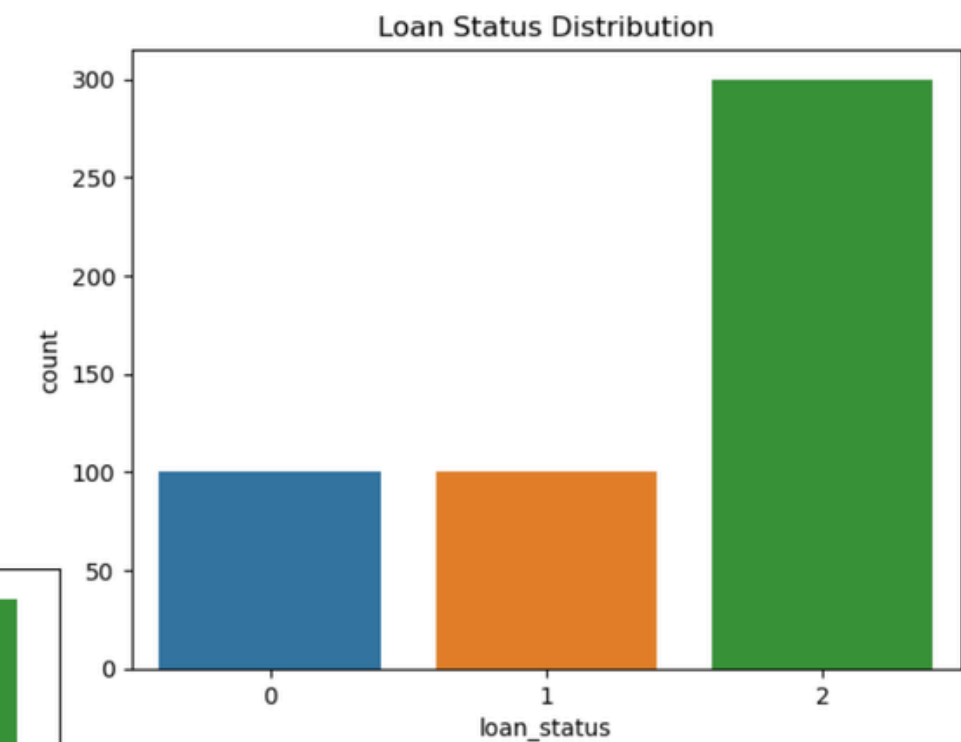
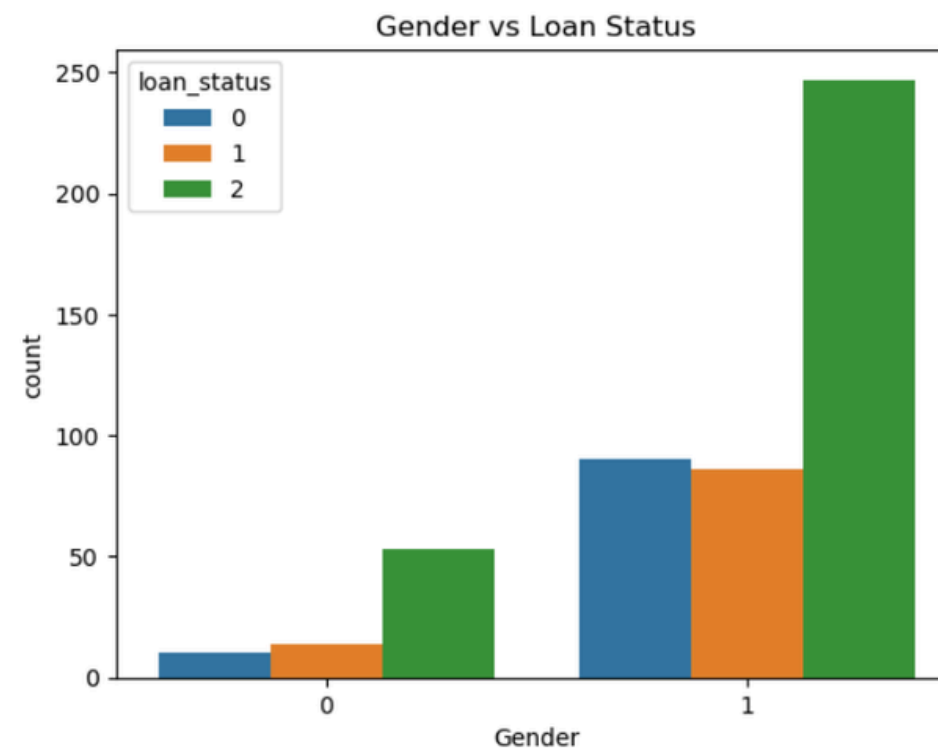
DATA VISUALIZATION

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.countplot(x='loan_status', data=df)
plt.title("Loan Status Distribution")
plt.show()
```

```
sns.countplot(x='Gender', hue='loan_status', data=df)
plt.title("Gender vs Loan Status")
plt.show()
```

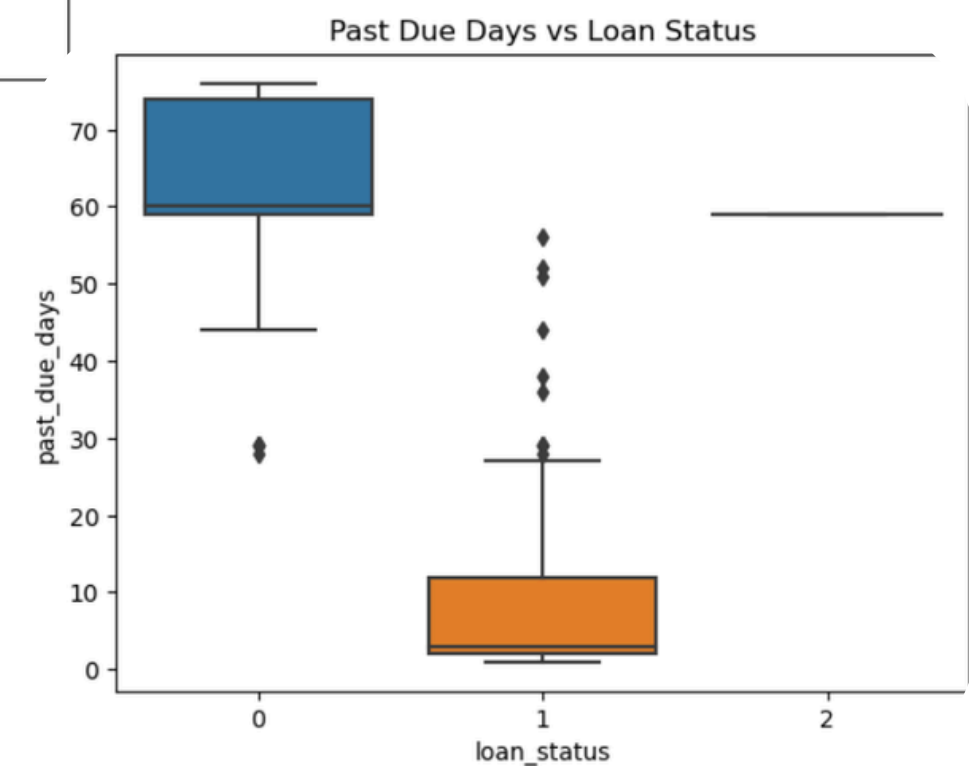
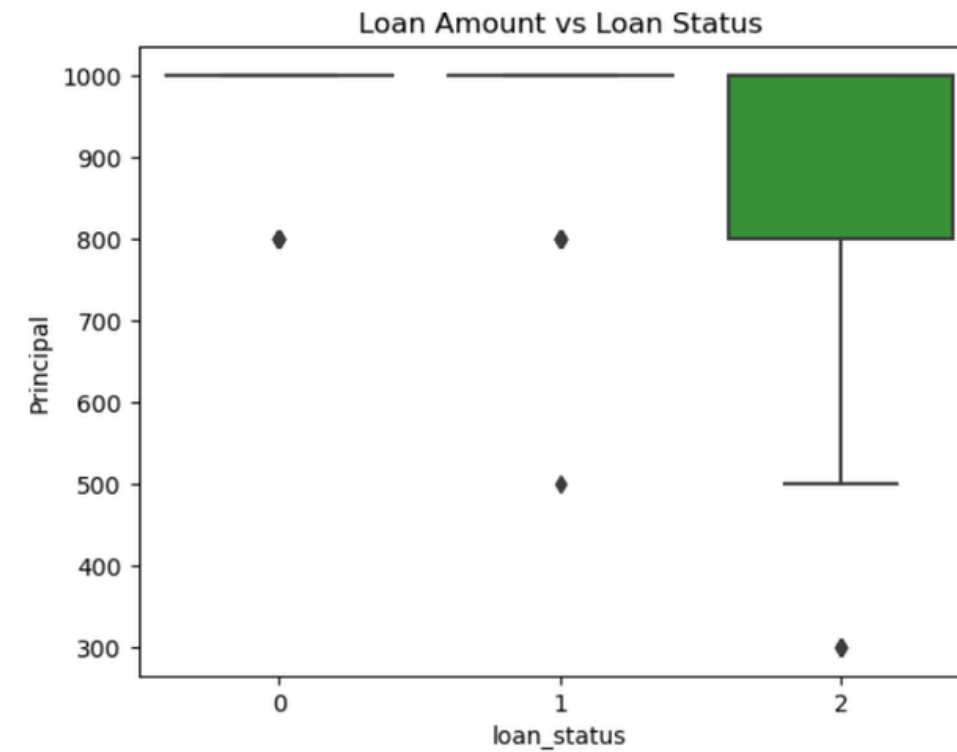
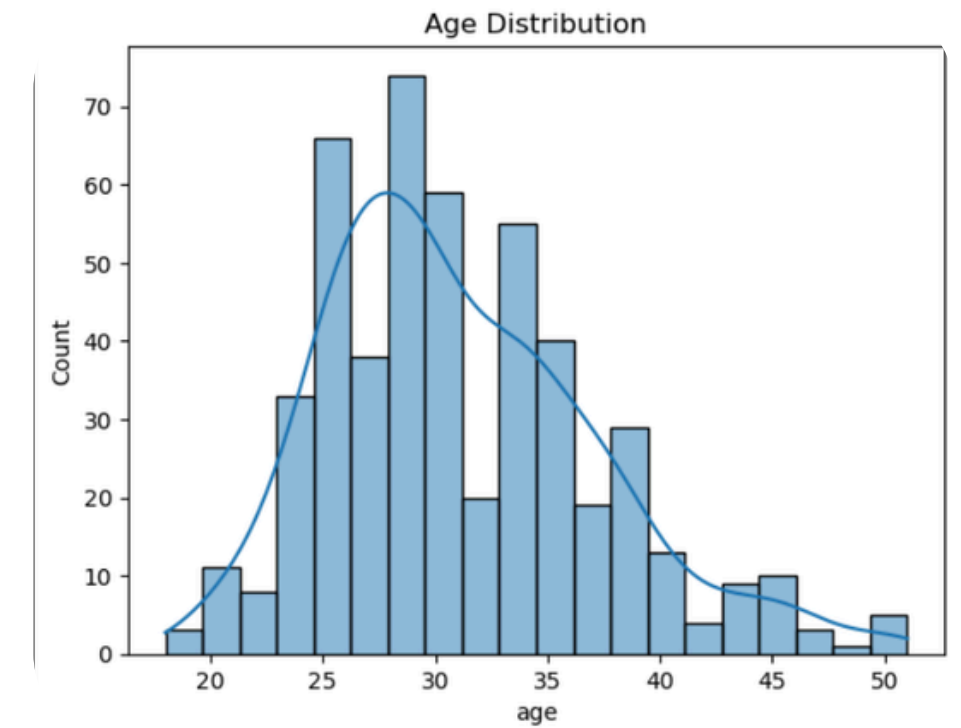
```
sns.countplot(x='education', hue='loan_status', data=df)
plt.title("Education vs Loan Status")
plt.show()
```



```
sns.histplot(df['age'], bins=20, kde=True)
plt.title("Age Distribution")
plt.show()
```

```
sns.boxplot(x='loan_status', y='Principal', data=df)
plt.title("Loan Amount vs Loan Status")
plt.show()
```

```
sns.boxplot(x='loan_status', y='past_due_days', data=df)
plt.title("Past Due Days vs Loan Status")
plt.show()
```



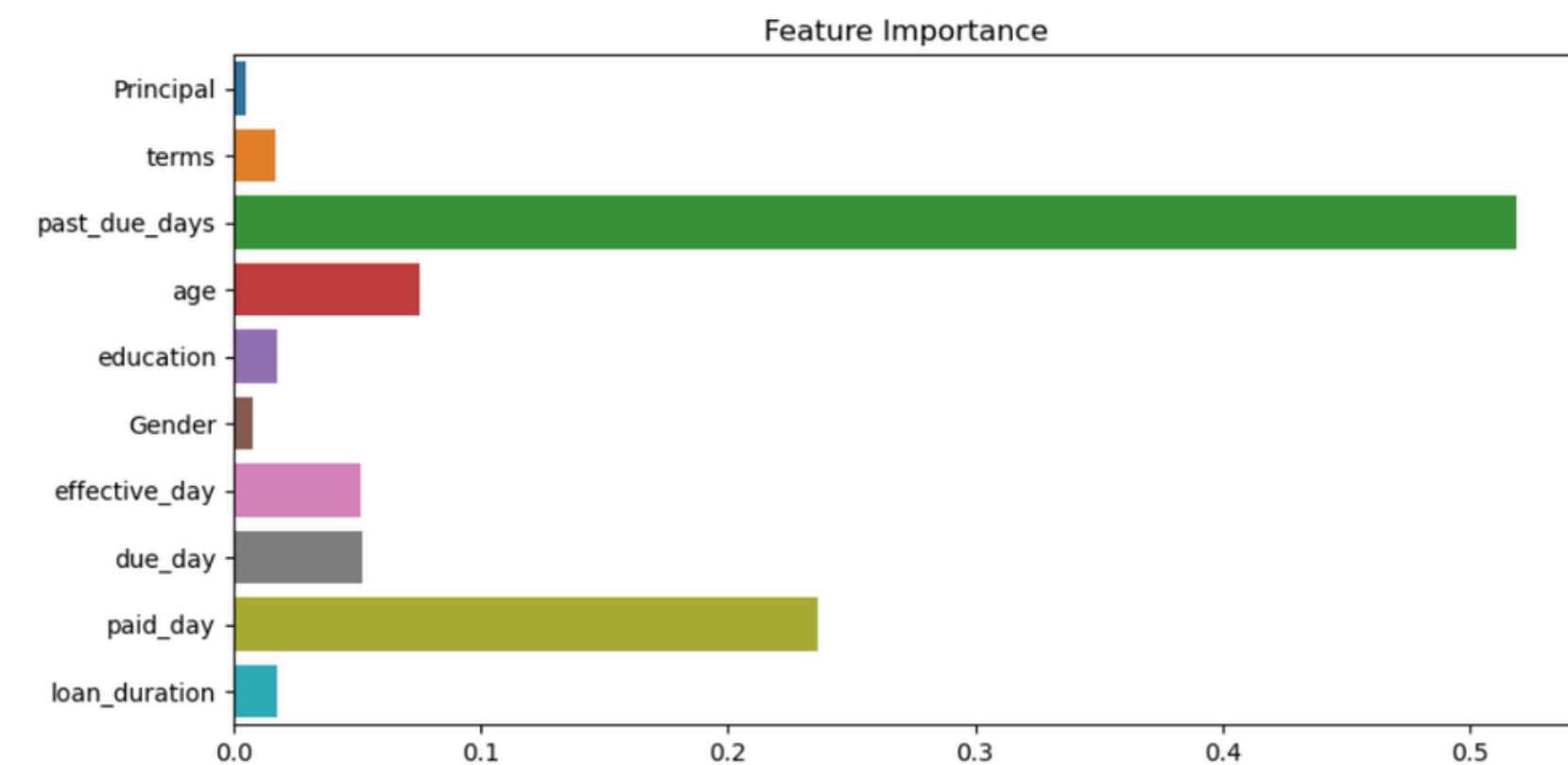
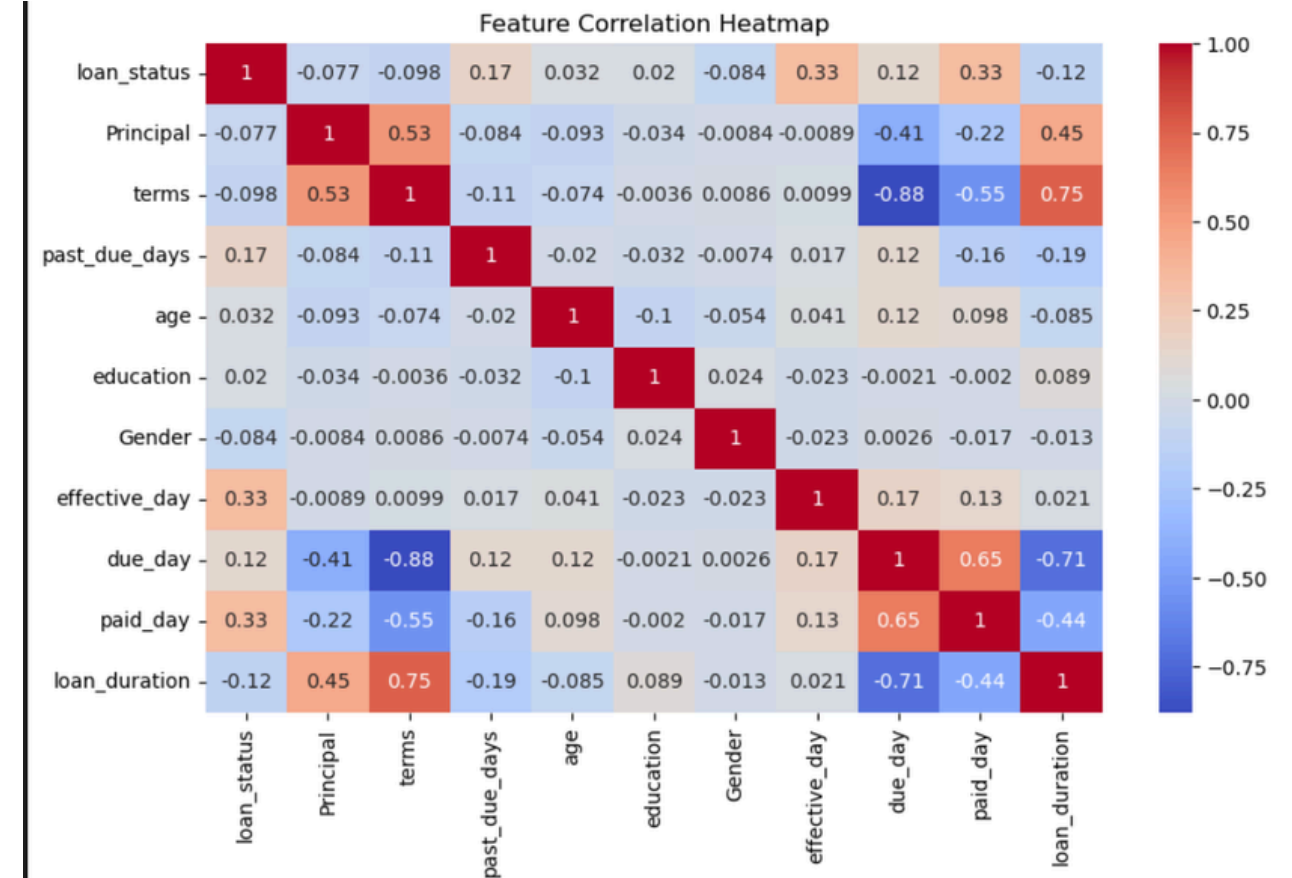
```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```

```
importances = rf.feature_importances_
features = X.columns
```

```
plt.figure(figsize=(10,5))
sns.barplot(x=importances, y=features)
plt.title("Feature Importance")
plt.show()
```



RESULTS & CONCLUSION

- Achieved ~70-85% accuracy. Key factors include loan duration and past due days. Model helps in better decision-making and reducing risk.

THANK YOU
